

Manuale operativo — claudiocammarano.com

Sito statico Eleventy (v3). Sorgenti in `src/`, output compilato in `_site/`. Deploy automatico su Netlify a ogni `git push` sul branch `main`.

Stack in due righe

Pezzo	Tecnologia
Generatore	Eleventy 3 + Nunjucks
Stile	CSS custom (<code>src/css/style.css</code>)
Deploy	Netlify (auto da git push)
Grafo concetti	Layout a forze custom in JS puro (nessuna dipendenza esterna, build-time + client-side vanilla)
Font	Source Serif 4 (Google Fonts)
Ricerca	Pagefind (<code>npm run index</code>)

Struttura cartelle

```
src/
  writings/      # Saggi originali (.md)
  curated/       # Link esterni con commento (.md)
  learning/      # Note di apprendimento (.md)
  lab/           # Note di ricerca in progress, IT/EN (.md) – output su /episteme-advisory/lab/, vedi
sezione 3
  _data/
    clusters.js      # Temi → Argomenti (tassonomia a 5 cluster)
    conceptsIndex.js # Tassonomia Concetti (fonte di verità)
    curatedTagAliases.js # Tag liberi dei curated → Argomento canonico
    tagDescriptions.json # Descrizioni degli Argomenti (facoltative)
    temiDescriptions.json # Descrizioni dei Temi/cluster (facoltative)
    seriesDescriptions.json
    graphLayout.js   # Calcola il layout del grafo concetti – generato, non a mano
  _includes/
    layouts/
      base.njk       # Layout base (header, footer)
      article.njk    # Layout saggi
      lab-note.njk   # Layout note /lab/ – versione leggera di article.njk
    components/      # Header, footer ecc.
  css/
    style.css        # Unico file CSS
  images/            # Immagini articoli
  downloads/         # File scaricabili del Learning Log (risorse + documenti del Lab, vedi sezione 3)
  graph-data.njk     # Endpoint statico /graph-data.json letto dal grafo (generato)
  mappa.njk          # Pagina /mappa/ – grafo completo concetti
  *.njk              # Pagine indice (temi, indice, curated, serie...)
```

Comandi

```
# Sviluppo locale con live reload
cd ~/Documents/PROJECTS/claudio-cammarano-site
npm start
# → http://localhost:8080

# Build di produzione
npm run build

# Deploy
git add -A
git commit -m "descrizione"
git push
# Netlify fa il resto in automatico
```

Importante: lanciare sempre `npm run build` (o `npm start`) prima di un push importante e leggere l'output in console. Da poco il build segnala in chiaro eventuali concetti citati ma non registrati (vedi sezione 5) — un controllo che prima non esisteva.

1. Pubblicare un saggio (writing)

File

Posizione: `src/writings/YYYY-MM-DD-slug-italiano.md`

Slug: titolo in minuscolo, spazi → trattini, accenti rimossi (è→e, à→a ecc.), max 6 parole.

Frontmatter minimo

```
---
layout: layouts/article.njk
title: "Titolo completo"
date: 2026-06-19
description: "1-2 frasi che funzionano da pitch, non da riassunto."
category: ["AI", "Geopolitica"]
lang: "🇮🇹 Italiano"
tags: [writings]
---
```

Campi opzionali

```
og_image: "/images/nome-file.avif"      # immagine Open Graph e hero
series: "Nome serie, I"                  # formato esatto con virgola + numero romano
english_version: "https://substack.com/..." # URL versione inglese
serie_totale_prevista: 3                 # solo se la serie ha lunghezza pianificata
```

Tag `category` — i valori "storici" dei writings

Questi sono i valori usati finora dai saggi (maiuscola compresa). L'elenco completo degli Argomenti del sito — molto più ampio, alimentato anche dai curated — è in `src/_data/clusters.js` (sezione 7).

Epistemologia & AI: `AI` · `Epistemologia` · `Filosofia` · `Scrittura` · `Vibe Coding` · `Claude`

Geopolitica & potere: `Democrazia` · `Geopolitica` · `Medio Oriente` · `Mediterraneo` · `Regimi politici` · `Scenario Planning` · `Teoria dei giochi`

Editoria & comunicazione: `Comunicazione` · `Dual Use` · `Formazione` · `Scienze della Comunicazione`

Italia & istituzioni: `Bologna` · `Humanities` · `Istituzioni` · `Italia` · `Politica`

Un saggio può in linea di principio usare anche uno qualsiasi degli Argomenti nati dai curated (es. `Strategia`, `Media`, `Economia`) — basta scriverlo esattamente come compare in `clusters.js`.

Struttura corpo

```
<!-- Immagine hero (opzionale) -->
<figure class="article-hero">
  
  <figcaption>Didascalia. © Fonte</figcaption>
</figure>

Paragrafo di apertura...

## Prima sezione

Testo. **Assertzioni chiave in grassetto come frasi intere.**
Titoli libri e riviste in *corsivo*.
[Link contestuale](https://url.com) integrato nel testo.

## Seconda sezione

Tabella markdown:

| Colonna A | Colonna B |
|---|---|
| Valore | Valore |

---

## Bibliografia essenziale

Cognome, N. (Anno). *Titolo*. Città: Editore.
```

Shortcode disponibili

```
{% kicker "ETICHETTA BREVE" %}

{% pullquote "Citazione lunga in evidenza con bordo sinistro." %}

{% figure "/images/nome.jpg", "Didascalia figura" %}
```

Info-box (sfondo grigio, bordo blu sinistro):

```
<div class="info-box">
  <p>Contenuto nota.</p>
</div>
```

Formule matematiche (LaTeX)

Inserire prima del corpo dell'articolo:

```
<script>
window.MathJax = {
  tex: { inlineMath: [['$', '$']], displayMath: [['$$', '$$']] },
  svg: { fontCache: 'global' }
};
</script>
<script id="MathJax-script" async src="https://cdn.jsdelivr.net/npm/mathjax@3/es5/tex-mml-ctml.js"></script>
```

Poi nel testo: `$formula inline$` oppure `$$formula display$$`.

Collegare i concetti — passaggio manuale, non automatico

A differenza dei **curated**, per i **writings** il collegamento ai concetti non passa dal frontmatter del file: vive in `src/_data/conceptsIndex.js`. **Salvare il file `.md` non basta.**

- Se l'articolo cita un concetto già in tassonomia → aggiungi `{ title: "Titolo articolo", url: "/writings/slug-articolo/" }` all'array `articles` della voce corrispondente in `conceptsIndex.js` (sezione 5 per il formato).
- Se introduce un concetto nuovo → crea una nuova voce.

La skill pubblica-articolo non fa questo passaggio. Genera solo il file markdown del writing; non tocca `conceptsIndex.js`.
Va fatto a mano, articolo per articolo.

Checklist completa per pubblicare un writing:

1. Salva il file in `src/writings/`
2. Apri `src/_data/conceptsIndex.js` e aggiorna le voci dei concetti citati (vedi sopra)
3. `npm run build` in locale — leggi l'output: se un concetto non è registrato, ora il build te lo segnala (sezione 5)
4. `git push` — non esiste un comando separato di "aggiornamento semantico": indici, pagine `/concetti/`, chip e grafo si rigenerano da soli al build, una volta che i dati sono corretti

2. Pubblicare un curated

File

Posizione: `src/curated/YYYY-MM-DD-slug-fonte-tema.md`

Convenzione slug: include spesso nome fonte + tema, es. `2026-04-09-nyt-google-ai-overviews.md`

Frontmatter

```
---
title: "Titolo originale dell'articolo esterno"
external_url: "https://url-completo.com/articolo"
source: "Nome Autore / Nome Pubblicazione"
date: 2026-06-19
description: "2-4 frasi in prima persona. Non riassunto neutro: commento di merito con punto di vista. Spiega perché vale la pena leggere e come si incastra con i temi del sito."
tags: [curated, ai, geopolitica]
concepts: ["Anthropic", "allineamento AI"]
---
```

`tags` : sempre inizia con `curated` , poi 1-3 tag tematici in minuscolo libero.

`concepts` : lista di nomi dalla tassonomia concetti (sezione 5). Usa `[]` se nessuno si applica. **I nomi sono case-sensitive** e devono coincidere esattamente con un `name` in `conceptsIndex.js` — altrimenti vengono scartati (vedi sezione 5 per il controllo automatico).

I `tags` dei curated ora contano come Argomenti — ma solo se sono "riconosciuti"

I tag liberi che scrivi in `tags` : (oltre a `curated`) vengono **tradotti** in un Argomento vero — visibile in `/indice/` , `/temi/` e con una pagina `/tag/...` propria — tramite la mappa in `src/_data/curatedTagAliases.js` . Funziona così:

- Se il tag che scrivi (case-insensitive) è già una chiave in `curatedTagAliases.js` → l'articolo conta per l'Argomento corrispondente. Esempio: scrivi `ai` minuscolo, l'articolo compare sotto l'Argomento `AI` .
- Se il tag **non** è in quella mappa → resta puramente decorativo: non genera nessuna pagina, non compare in nessun indice. Non è un errore, ma è facile non accorgersene.

Quando introduci un tema nuovo nei curated, controlla `curatedTagAliases.js` . Se il tag non c'è:

1. Decidi il nome canonico dell'Argomento (Title Case, in italiano per coerenza con gli altri — es. `Sorveglianza` , non `surveillance`).
2. Aggiungi la riga `"tuo-tag-minuscolo": "Nome Canonico"` in `curatedTagAliases.js` .

3. Aggiungi "Nome Canonico" all'array del cluster giusto in `src/_data/clusters.js` (sezione 7) — altrimenti l'Argomento esiste ma non ha un Tema e non appare in `/temi/`.
4. Facoltativo: aggiungi una descrizione in `tagDescriptions.json`.

Se invece il tema è in realtà un'entità (una persona, un paese, un'istituzione) e non un argomento trasversale, probabilmente appartiene ai Concetti (sezione 5), non agli Argomenti — è il caso tipico dei nomi di paese: vanno nel campo `concepts:`, non in `tags:`.

Body

Opzionale. Si usa solo per commenti molto estesi. Di solito il frontmatter basta.

3. Pubblicare una nota del Lab

Solo in inglese, per scelta

Il pubblico della sezione (revisori, potenziali endorser arXiv, target journal) legge solo in inglese, ed è la lingua in cui si cerca il cluster semantico su cui punta il GEO del progetto (*ontological taxonomy*, *inter-annotator agreement* ecc.). Il layout `lab-note.njk` imposta `lang: en` di default — non va specificato nel frontmatter delle singole note, né serve gestire coppie di file o link incrociati.

URL — non è `/lab/`

La sezione vive sotto `/episteme-advisory/lab/`, non come sezione di primo livello (per non affollare la nav principale). L'URL è impostata via `permalink`, non dalla posizione dei file: - `src/lab.njk` ha `permalink: "/episteme-advisory/lab/"` in frontmatter - `src/lab/lab.11tydata.js` è un directory data file che assegna a ogni nota `permalink: "/episteme-advisory/lab/<nome-file-senza-estensione>/"`, data compresa nello slug (Eleventy la toglierebbe di default da `fileSlug / filePathStem` — per questo serve lo script invece di un semplice `.json`)

I file sorgente restano comunque in `src/lab/` — solo l'URL di output cambia. Non serve toccare `src/episteme-advisory.njk` per la struttura; su quella pagina c'è solo un blocco `.episteme-reading` che rimanda al Lab.

Niente retrodatazioni. Ogni nota va datata il giorno reale in cui viene pubblicata — non si simula una cronologia pregressa. Se serve raccontare il lavoro già fatto prima di aprire il log, si scrive una nota "stato dell'arte"/"project overview" datata oggi (vedi il primo esempio in `src/lab/`), non una serie di note con date fittizie nel passato.

File

Posizione: `src/lab/YYYY-MM-DD-slug.md`.

Frontmatter

```
---
layout: layouts/lab-note.njk
title: "Note title"
date: 2026-07-03
---
```

Campi opzionali

```
project: "Post-cognition"      # a quale filone di ricerca appartiene – usalo solo se ce n'è più di uno
stage: "Writing & submission"  # a che punto del processo si trova QUESTA nota – vedi tassonomia sotto
discipline: ["Epistemology", "Statistics"]  # a quali campi del sapere attinge – non visibile, solo JSON-LD
method: ["Quantitative analysis"]  # con che tipo di evidenza/tecnica lavora – non visibile, solo JSON-LD
description: "1-2 sentences"  # solo se serve per meta/OG, non obbligatoria
```

Niente campo `tags`. Le note del Lab non hanno etichette libere in testa: con un solo filone di ricerca alla volta sarebbero quasi sempre identiche da una nota all'altra, e non essendo linkate a `/tag/slug/` non aggiungerebbero navigazione reale — solo rumore.

Tassonomia del Lab — separata da quella del sito

Tre assi indipendenti, pensati apposta per non mescolarsi con `clusters.js` / `conceptsIndex.js` / Indice (che si riempirebbero di "Epistemologia" ripetuta su ogni nota). Nessuna registrazione centrale, nessun controllo automatico — sono liste aperte, aggiungi un valore quando serve:

- **discipline** (lista) — campo/i del sapere. Partenza: *Epistemology, Philosophy of language, Computational linguistics / NLP, Statistics, Cognitive science, AI governance & regulation*.
- **stage** (valore singolo) — fase del processo in cui si trova la nota. Partenza: *Framework design, Taxonomy development, Corpus construction, Empirical validation, Data analysis, Writing & submission, Dissemination*.
- **method** (lista) — tipo di evidenza/tecnica. Partenza: *Qualitative coding, Quantitative analysis, Statistical modeling, Model evaluation / benchmarking, Literature review, Annotation / inter-annotator agreement*.

Solo **stage** è visibile (badge in testata, nella card dell'indice, nella striscia home). **discipline** e **method** restano metadati strutturati nel JSON-LD (`about` / `keywords`) — segnale per la ricerca semantica/GEO senza aggiungere pillole in pagina.

Cosa NON fa questo layout (di proposito)

Niente abstract obbligatorio, niente serie, niente immagine hero, niente TOC, niente share bar, niente post correlati, niente tag liberi. `project` / `stage` sono etichette visive che non toccano `clusters.js` né `conceptsIndex.js`. Le date si formattano con il filtro `formatDateEN` (inglese), non `formatDate` (italiano, usato dal resto del sito). Il corpo della nota (`.article-full-lab` `.article-body`) è a 16px invece dei 18px dei writings — restano note di cantiere, non saggi.

Visibilità — niente voce in nav

Il Lab non ha una voce propria nel menu di primo livello (troppo affollato) e non ha nessun'altra presenza fissa in navigazione. È raggiungibile in due punti:

- un riquadro blu cliccabile (`.episteme-lab-cta`) sotto il paragrafo "Il laboratorio" in `src/episteme-advisory.njk`
- una striscia blu in home (`.lab-banner` in `src/index.njk`), subito sotto il primo saggio in evidenza, che mostra **solo l'ultima nota pubblicata** (`collections.lab[0]`) — si aggiorna da sola, non richiede manutenzione quando esce una nota nuova

Documenti citati — link automatico e Learning Log

Le note del Lab citano spesso un documento di lavoro (tassonomia, coding manual, corpus, script, bibliografia). Questi documenti sono anche scaricabili dal Learning Log (`/learning/`), in una sezione verde separata "Dal Lab" — colore Episteme Advisory (`#0F6E56`), diverso apposta dal blu delle risorse standard del Learning Log, per segnalare che sono materiali di lavoro grezzi, non risorse finite.

Il meccanismo è tutto nel frontmatter — non si tocca `learning.njk` né si crea alcun file in `src/learning/`.

1. Il file fisico va in `src/downloads/` (stessa cartella piatta delle risorse esistenti, copiata in automatico da `addPassthroughCopy` in `.eleventy.js`). Formato preferito: **PDF**, anche per documenti nati come testo — evita ambiguità di formato ed è leggibile ovunque senza Word. Script (`.py`) e dati strutturati (`.bib` , `.json`) restano nel loro formato nativo. Se un file contiene path assoluti della propria macchina (es. `/Users/nome/Documents/...`), toglierli prima di caricarlo — sono superflui per chi scarica e rivelano struttura di cartelle privata.

2. Nel frontmatter della nota che introduce il documento, aggiungere un blocco `documents:` :

```
documents:
- file: "nome_file.pdf"
  label: "Titolo leggibile del documento"
  version: "2.0"           # facoltativo — numero versione, o testo libero ("v4", "Addendum a v1.0")
  id: "id-stabile"        # facoltativo — solo se questo documento SOSTITUISCE una versione precedente
                           # già registrata altrove (vedi sotto)
```

Questo blocco va **solo sulla nota che introduce il documento** (di norma quella dedicata, non le note successive che lo ricitano di sfuggita). Genera in automatico: il link nel testo (via shortcode, punto 3), la card verde nel Learning Log con data (quella della nota), versione e link "← Torna alla nota".

3. Nel corpo della nota — di QUALSIASI nota, non solo quella che introduce il documento — sostituire la menzione del file con lo shortcode:

```
Document: {% labdoc "nome_file.pdf" %}
```

Lo shortcode cerca il filename in tutti i `documents:` dichiarati nel Lab e genera il link di download. Se il file non è ancora registrato in nessun frontmatter, torna al vecchio stile testuale (``nome_file.pdf``) invece di rompere la build — un promemoria visivo che manca il passo 2.

4. Versioni che sostituiscono un documento precedente. Se una nota pubblica una nuova versione di un documento già registrato (es. una Addendum viene poi assorbita in un Coding Manual v2.0), dare ai due blocchi `documents:` lo **stesso id**. Il Learning Log mostra automaticamente solo la versione più recente per ogni `id` (per data della nota); quella precedente sparisce dalla griglia "Dal Lab" ma resta scaricabile dal suo URL e dal link `{% labdoc %}` nella nota vecchia — nulla si cancella, cambia solo cosa compare in vetrina. Senza `id`, ogni documento ha la propria card indipendente (il caso normale).

Licenza: tutte le card "Dal Lab" mostrano CC BY-NC-SA 4.0, la stessa del resto del Learning Log — non è un campo per documento, è fissa nel template.

Tre ricette pratiche:

- *Nuova nota che cita un documento mai visto prima* → aggiungere `documents:` con i suoi dati, mettere il file in `src/downloads/`, usare `{% labdoc %}` nel testo.
- *Nuova nota che ricita un documento già registrato* → nessun `documents:` da aggiungere, solo `{% labdoc "nome_file.pdf" %}` nel testo (il link si risolve da solo).
- *Nuova nota che pubblica una versione aggiornata di un documento esistente* → `documents:` con lo stesso `id` del documento precedente; il file vecchio non si tocca né si cancella.

4. Aggiungere una serie

1. I saggi di una serie usano il campo `series: "Nome serie, I"` (con numero romano).
2. Aggiungere la descrizione della serie in `src/_data/seriesDescriptions.json`:

```
{
  "Nome serie": "Descrizione della serie per la pagina /serie/."
}
```

1. Nessun altro intervento: le pagine `/serie/` e `/serie/nome-serie/` si generano automaticamente.

5. Tassonomia concetti (`_data/conceptsIndex.js`)

Questo file è la **fonte di verità** per l'indice analitico, le pagine `/concetti/slug/` e il grafo. È il livello più specifico dei tre (Temi → Argomenti → **Concetti**): persone, teorie, testi, istituzioni, luoghi e paesi citati con peso argomentativo.

Struttura di una voce

```
{
  name: "Axelrod, Robert",      // nome esatto, usato come ID
  type: "persona",              // persona | teoria | testo | istituzione | luogo | paese
  articles: [                   // articoli writings che lo citano
    { title: "L'ombra del futuro", url: "/writings/2026-04-15-lombra-del-futuro/" }
  ],
  note: "Testo facoltativo."    // nota descrittiva (opzionale)
}
```

Aggiungere un nuovo concetto

1. Aprire `src/_data/conceptsIndex.js`
2. Aggiungere la voce nell'array, rispettando il formato sopra
3. Per i **writings**: inserire subito gli articoli che lo citano nell'array `articles`

4. Per i **curated**: lasciare `articles: []` e aggiungere il nome nel campo `concepts:` del file `curated` — il sistema li unisce automaticamente a build time
5. Fare il build: la pagina `/concetti/nome-slug/` appare automaticamente

Tipi validi — attenzione a `paese` VS `luogo`

Tipo	Esempi
<code>persona</code>	Axelrod, Robert · Trump, Donald · Arendt, Hannah
<code>teoria</code>	scenario planning · dual use · allineamento AI
<code>testo</code>	The Evolution of Cooperation · Antifragile
<code>istituzione</code>	Anthropic · Palantir · DARPA · Studio Ghibli
<code>luogo</code>	Bologna · Libano / Beirut · Taiwan · TSMC
<code>paese</code>	Europa · Stati Uniti · Cina · Russia · Giappone

Ogni nome di stato/nazione è `paese`, non `luogo` — anche quando nel linguaggio comune lo chiameremmo "un luogo" (es. Giappone, Germania, Iran). `luogo` è riservato a città, regioni, snodi geografici specifici che non sono uno stato (Bologna, Beirut, Taiwan come territorio conteso). I due tipi finiscono in sezioni diverse dell'indice analitico ("Paesi" e "Luoghi"): se un nome non si trova dove lo cerchi, controlla prima l'altra sezione.

Il controllo automatico — niente più concetti persi in silenzio

Per anni il sistema ha scartato **senza nessun avviso** qualsiasi nome in `concepts:` di un `curated` che non corrispondesse a una voce già presente in `conceptsIndex.js`. Risultato: articoli pubblicati con concetti che restavano invisibili nell'indice e nel grafo, senza che nessuno se ne accorgesse — a volte per mesi.

Ora il build segnala il problema in console. Ogni volta che esegui `npm run build` o `npm start`, se un `curated` cita un concetto non registrato compare un blocco come questo:

```
⚠ CONCETTI NON REGISTRATI (ignorati nell'indice/grafico) — aggiungili a src/_data/conceptsIndex.js:
./src/curated/2026-06-21-esempio.md -> "Nome Concetto"
```

Se lo vedi: apri `conceptsIndex.js`, aggiungi la voce mancante (sezione precedente), rifai il build. L'avviso scompare quando tutto è registrato. **Su Netlify lo stesso messaggio compare nei log di deploy** (Deploy → log) — quindi anche pubblicando solo via `git push`, senza build locale, il problema resta visibile.

6. Come funziona la navigazione semantica

Il sito ha tre livelli di navigazione, dal più ampio al più specifico:

```
TEMI (5 cluster)           /temi/
  ↓ ogni tema raggruppa più Argomenti
ARGOMENTI (~49 tag)       /tag/slug/
  ↓ ogni Argomento elenca gli articoli (writings + curated) che lo usano
CONCETTI (persone, teorie...) /concetti/slug/
  ↓ entità citate con peso argomentativo dentro gli articoli
```

Più l'**indice analitico** (`/indice/`), che mostra tutti e tre i livelli insieme in un'unica pagina — pensato apposta perché un visitatore capisca subito su quale livello si trova.

Temi (`/temi/`) e Argomenti (`/tag/slug/`)

- I **Temi** sono i cluster definiti in `clusters.js` (sezione 7): 5 raggruppamenti larghi.
- Gli **Argomenti** sono i singoli tag — quelli storici dei writings (`category:`) e quelli più recenti emersi dai curated (`tags:` , tradotti via `curatedTagAliases.js` , sezione 2).
- Ogni Argomento ha una pagina `/tag/slug/` che mostra **sia** i writings che lo usano in `category:` **sia** i curated il cui tag si traduce in quell'Argomento — stessa pagina, card diverse per tipo.

- Le descrizioni (opzionali) vivono in `tagDescriptions.json` (per gli Argomenti) e `temiDescriptions.json` (per i Temi).

Concetti (`/concetti/slug/`)

Ogni concetto ha una pagina propria con: i "concetti vicini" (chip cliccabili — concetti che condividono almeno un articolo), la lista degli articoli collegati, e una **finestra sul grafo globale** — una porzione ritagliata del grafo completo, centrata sul concetto, con una minimappa che mostra dove si trova rispetto all'insieme. Link "Vedi mappa completa →" per aprire `/mappa/` centrata sullo stesso nodo.

Il grafo è **uno solo**, calcolato una volta a build time da `graphLayout.js` (layout a forze custom, nessuna libreria esterna) e esposto come JSON statico su `/graph-data.json`. Ogni pagina concetto e la pagina `/mappa/` lo leggono e ne mostrano porzioni diverse — non sono grafi separati.

Il grafo è scope-limitato a Concetti e articoli. Temi e Argomenti non ci entrano: è una scelta deliberata, per non sovraccaricare una visualizzazione già densa.

Visibile solo da schermi ≥ 900px. Su mobile, sia le pagine concetto sia `/mappa/` mostrano la sola lista testuale — nessuna modifica al mobile rispetto a prima dell'introduzione del grafo a finestra.

Il flusso dei dati, in breve

- **Writings:** Argomento ← `category:` nel frontmatter. Concetto ← voce manuale in `conceptsIndex.js` (sezione 1).
- **Curated:** Argomento ← `tags:` nel frontmatter, tradotto via `curatedTagAliases.js` (sezione 2). Concetto ← `concepts:` nel frontmatter, deve corrispondere a una voce già in `conceptsIndex.js` (sezione 5).

Non esiste un comando per "aggiornare l'indice semantico". È lo stesso identico `git push` (o `npm run build`) con cui pubblichi il sito. Se i dati a monte sono corretti, il build successivo rigenera tutto: indici, pagine concetto, grafo, Temi, Argomenti — niente da "richiamare" a parte.

7. Temi e Argomenti (`_data/clusters.js`)

`clusters.js` è la tassonomia dei **Temi**: un oggetto che mappa ogni nome di Tema a un array di Argomenti.

```
module.exports = {
  "Epistemologia & AI":      ["Filosofia", "Epistemologia", "AI", "Scrittura", "Vibe Coding",
  "Claude", ...],
  "Geopolitica & potere":    ["Geopolitica", "Teoria dei giochi", "Difesa", "Europa", ...],
  "Editoria & comunicazione": ["Comunicazione", "Dual Use", "Media", "Editoria", ...],
  "Italia & istituzioni":     ["Italia", "Bologna", "Humanities", "Femminismo", ...],
  "Economia & lavoro":       ["Economia", "Lavoro"]
};
```

Cinque Temi in tutto (il quinto, "Economia & lavoro", è nato dai curated: nessuno dei 4 Temi originari aveva una casa naturale per quei contenuti).

Per aggiungere un Argomento a un Tema: inserirlo nell'array del Tema corrispondente. `/temi/`, `/indice/` e il breadcrumb si aggiornano al prossimo build.

Per i writings: basta scrivere il nome esatto in `category:` (sezione 1).

Per i curated: il tag in `tags:` deve prima essere tradotto in `curatedTagAliases.js` (sezione 2) — è un passaggio in più, perché i tag dei curated sono scritti liberi e minuscoli, non nel formato canonico.

Le descrizioni dei Temi vivono in `temiDescriptions.json`, quelle degli Argomenti in `tagDescriptions.json` — entrambe facoltative, entrambe scritte in prima persona e ancorate ai contenuti reali del sito, non enciclopediche.

8. Cosa NON toccare senza capire

File	Perché è delicato
<code>.eleventy.js</code>	Filtri e collections: un errore blocca il build
<code>src/_includes/layouts/article.njk</code>	Template di tutti i saggi
<code>src/_includes/layouts/base.njk</code>	Template base di tutto il sito
<code>src/css/style.css</code>	Unico file CSS, ~3800 righe
<code>src/_data/conceptsIndex.js</code>	Fonte di verità per indice e concetti
<code>src/_data/clusters.js</code>	Tassonomia Temi/Argomenti — usata da <code>/temi/</code> , <code>/indice/</code> , breadcrumb e dal layout del grafo
<code>src/_data/curatedTagAliases.js</code>	Traduce i tag liberi dei curated in Argomenti — un nome cambiato qui "spegne" silenziosamente quell'Argomento per i curated già pubblicati
<code>src/_data/graphLayout.js</code>	Calcola le posizioni del grafo. Non richiede modifiche manuali: si rigenera da solo leggendo <code>conceptsIndex.js</code> e <code>clusters.js</code>

9. Troubleshooting rapido

Un concetto/articolo che ho appena pubblicato non appare da nessuna parte → **Prima cosa da controllare:** lancia `npm run build` e leggi la console. Se il concetto è citato in un `concepts:` di un curated ma non registrato, ora compare un avviso `⚠ CONCETTI NON REGISTRATI` con il nome del file e del concetto mancante (sezione 5). Risolve la maggior parte dei casi.

Cerco un Concetto-paese e non lo trovo in "Luoghi" → I nomi di stato sono `paese`, non `luogo`: controlla nella sezione "Paesi" dell'indice (sezione 5).

Build fallisce con errore Nunjucks → Controllare la sintassi del file `.njk` modificato. Errore più comune: tag `{% %}` non chiusi.

Pagina concetto non appare pur risultando registrata → Verificare che il nome in `concepts:` del curated corrisponda **esattamente** (maiuscole comprese) a un `name` in `conceptsIndex.js`. Se non corrisponde, ricomparirà nell'avviso di build (sezione 5).

Chip concetti non appaiono in un articolo → Per i writings: verificare che l'URL dell'articolo sia presente nell'array `articles` della voce in `conceptsIndex.js`. → Per i curated: verificare che il campo `concepts:` sia presente e non vuoto.

Un tag di un curated non genera nessuna pagina `/tag/...` → Non è in `curatedTagAliases.js` (sezione 2): è decorativo, non un Argomento. Aggiungilo alla mappa se merita di diventarlo.

Netlify build fallisce → Guardare il log su `app.netlify.com` → Deploy → log del deploy fallito. Di solito è un errore di sintassi in un file `.md` o `.njk`, oppure (da oggi) un avviso di concetti non registrati che però non blocca il deploy — è solo un promemoria, non un errore.

Grafo non compare → Il grafo (pagine concetto e `/mappa/`) è visibile solo su schermo $\geq 900\text{px}$. Su mobile è sostituito dalla lista articoli, invariata.